

AUTOMATED COLOR – BASED PRODUCT SORTING SYSTEM FOR MANUFACTURING INDUSTRIES

ASSIGNMENT – 1

TEAM MEMBERS

- 1. JAGADEEP S – 24MER018**
- 2. JEEVA P – 24MER019**
- 3. JOGINDER S – 24MER021**
- 4. RAGUL R – 24MER046**

PROJECT OVERVIEW

The Automated Color-Based Product Sorting System is a smart automation project developed to automatically identify and separate products based on their color. Color-based sorting is widely useful in manufacturing industries for applications such as product inspection, packaging, quality control, and material handling. The main purpose of this project is to demonstrate how sensors, microcontrollers, actuators, and IoT technology can be integrated to perform an automated industrial task.

The system uses an ESP8266 NodeMCU as the central control unit. The ESP8266 is a Wi-Fi-enabled microcontroller that receives color information from the sensor, processes the information, and controls the output devices. Its built-in Wi-Fi capability makes it suitable for future IoT-based monitoring and data communication.

A TCS3200 color sensor is used to detect the color of the product. The sensor contains photodiodes that detect different color components of reflected light and converts the detected light intensity into electrical frequency signals. The ESP8266 reads these signals and determines the approximate color of the object. The sensor should be placed at a suitable distance and fixed position to obtain consistent readings.

When a product is placed in front of the sensor, the TCS3200 detects its color and sends the corresponding signal to the ESP8266. The controller compares the sensor readings with predefined values in the program and identifies the color. Based on the detected color, the ESP8266 activates the appropriate LED indicator. For example, when a red product is detected, the red LED turns ON, while a green product activates the green LED. This provides a simple visual indication of the detected product.

A servo motor is used as the sorting actuator. After identifying the color, the ESP8266 sends a control signal to the servo motor. The servo rotates to a predefined angle corresponding to the detected color. This movement represents the sorting action. In a complete industrial system, the servo could control a flap or mechanical diverter that directs products into different collection areas.

In the present prototype, a conveyor mechanism is not available, so the product is placed manually in front of the TCS3200 sensor. This method allows the main concept of color detection and automatic sorting to be demonstrated without a conveyor. The servo movement represents the direction in which the product would be sorted in an actual manufacturing system.

The accuracy of the system depends on several factors, including the lighting conditions, distance between the sensor and product, surface characteristics, product color, and sensor position. Proper calibration is therefore required before operation. The sensor should preferably be protected from strong external light so that the readings remain stable.

The ESP8266 also provides the possibility of connecting the system to an IoT platform through Wi-Fi. In an advanced version, information such as detected color, number of products, sorting status, and system operating condition can be transmitted to a dashboard. This allows the operator to monitor the sorting process remotely and collect production data.

PROBLEM STATEMENT

In manufacturing industries, manual identification and sorting of products based on color is a time-consuming process and can lead to human errors and inconsistent sorting. This can reduce production efficiency and increase the need for manual labor. Therefore, there is a need for a low-cost and reliable automated system that can identify products based on their color and perform the sorting operation with minimum human intervention.

The proposed project aims to design and develop an Automated Color-Based Product Sorting System using an ESP8266 NodeMCU and TCS3200 color sensor. The system will detect the color of the product, process the sensor information, and control a servo motor to represent the appropriate sorting action. The system will also provide visual indication using LEDs and can be further enhanced with IoT-based monitoring for industrial automation applications.

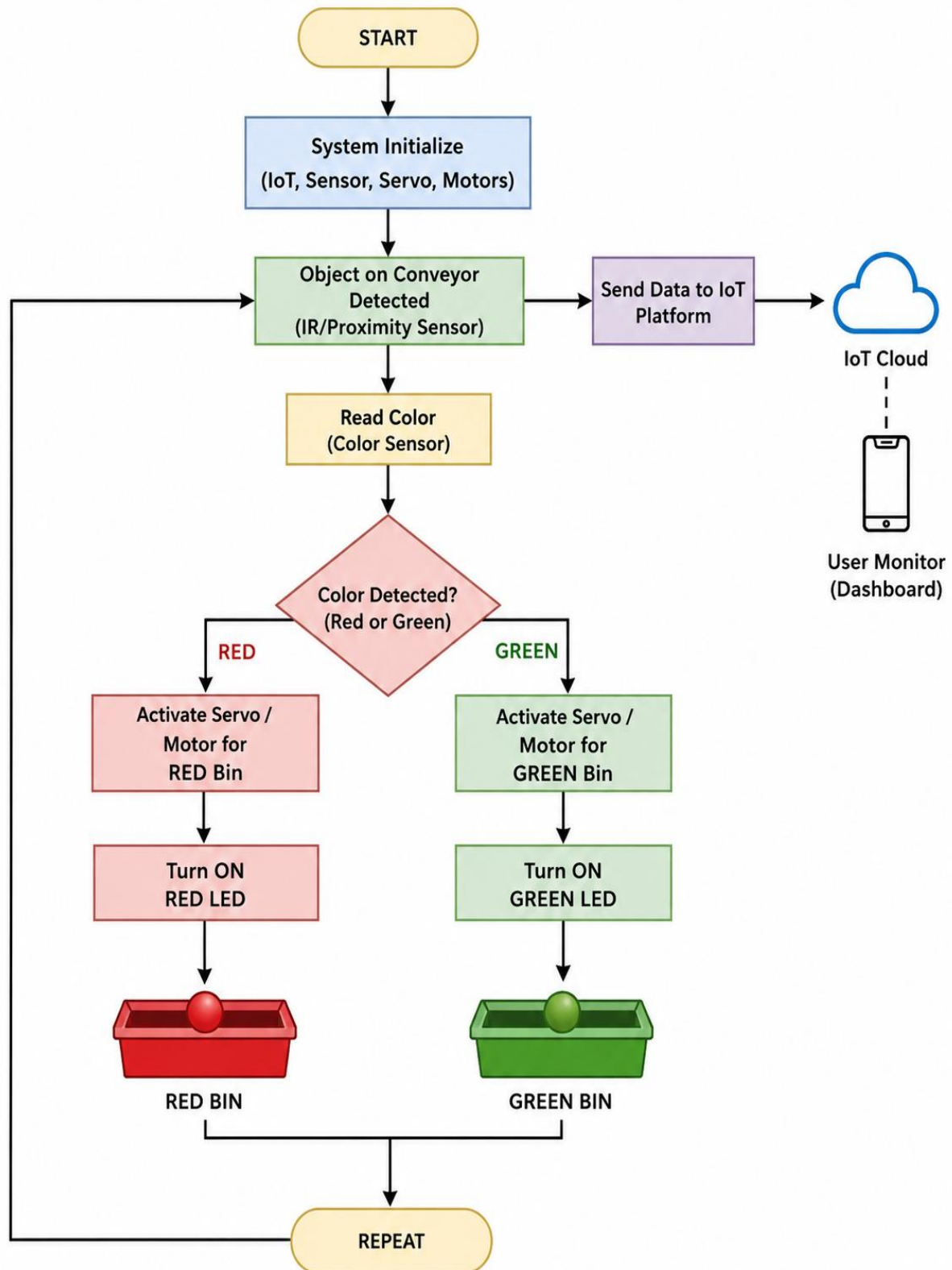
PROPOSED SOLUTION

The proposed solution is to develop an automated color-based product sorting system using an ESP8266 NodeMCU, TCS3200 color sensor, servo motor, and LED indicators. The TCS3200 sensor detects the color of the product and sends the sensor data to the ESP8266 for processing. The ESP8266 compares the detected color with predefined values and identifies the product category.

Based on the detected color, the ESP8266 activates the corresponding LED indicator and controls the servo motor to move to a specific position representing the sorting direction. In the current prototype, the product is manually placed in front of the sensor because a conveyor system is not available. The servo movement demonstrates how the product would be separated into different categories in an industrial system.

The ESP8266 also provides Wi-Fi connectivity, which can be used to monitor the detected colors, product count, and sorting status through an IoT platform. The proposed system reduces manual effort, improves sorting consistency, and provides a low-cost solution for demonstrating automated product sorting in manufacturing industries.

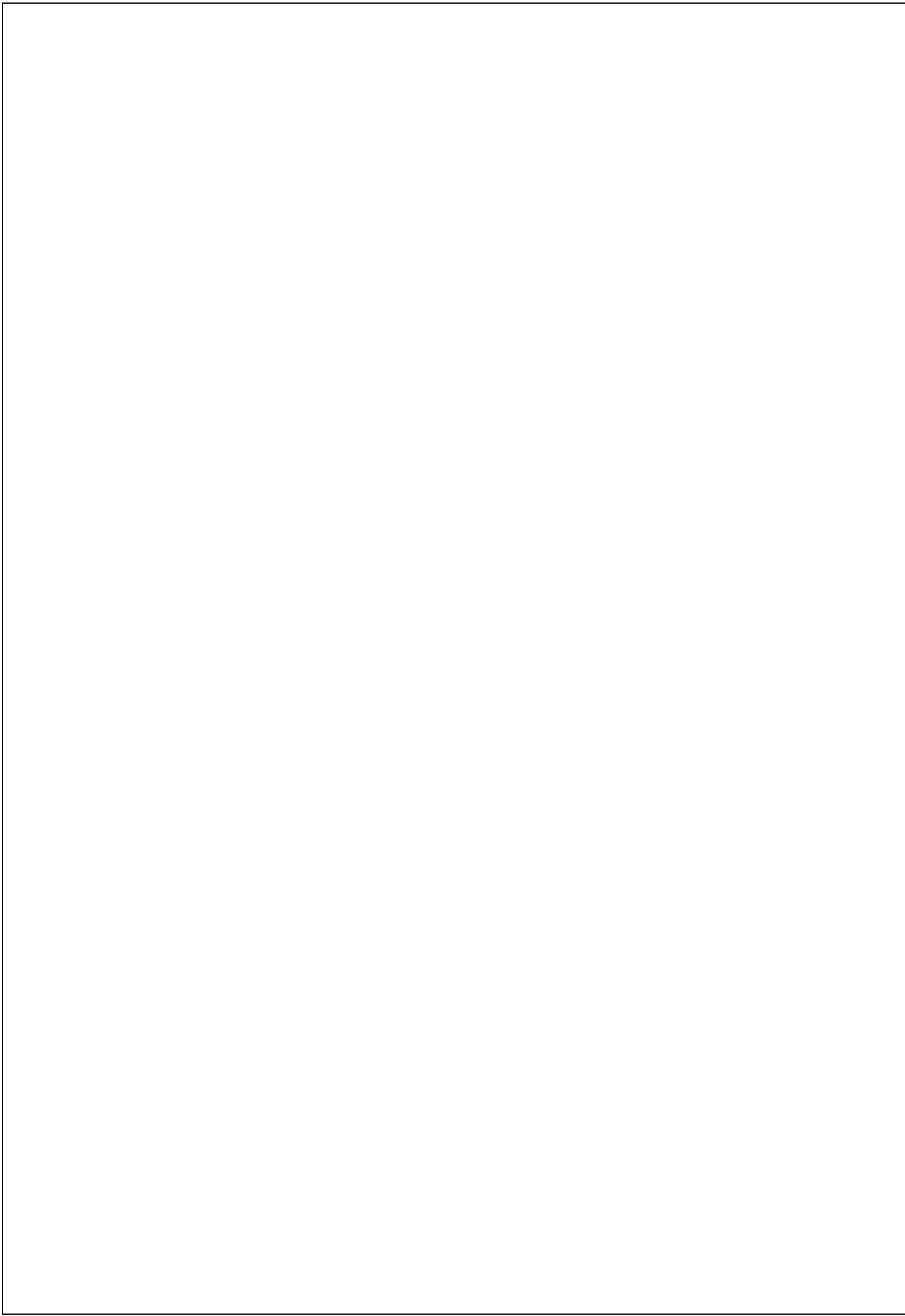
SYSTEM ARCHITECTURE



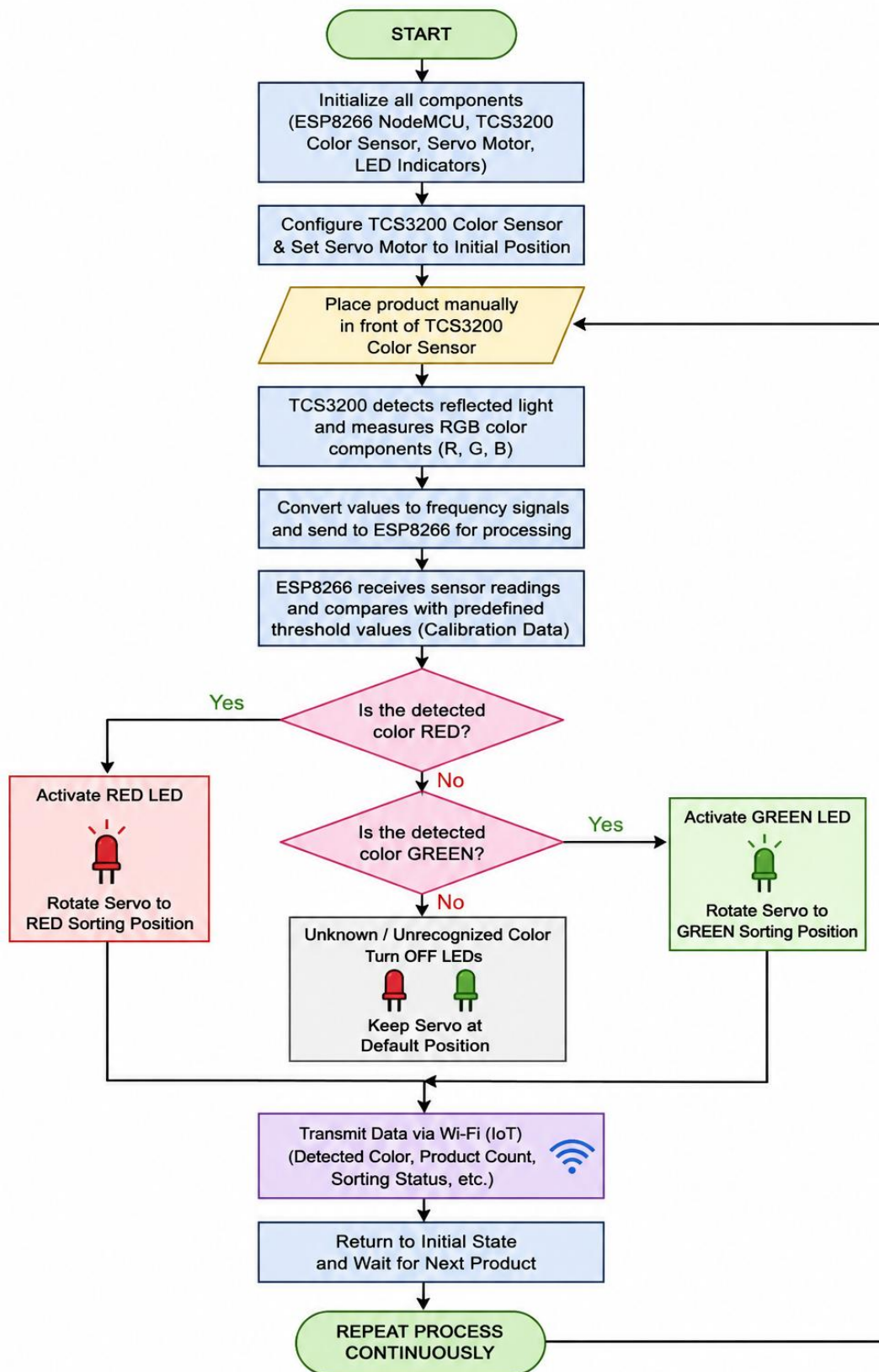
CIRCUIT TABLE

COMPONENT	COMPONENT PIN	ESP8266 NodeMCU
TCS3200 Color Sensor	S0, S1	D1, D2
TCS3200 Color Sensor	S2, S3	D5, D6
TCS3200 Color Sensor	OUT	D7
TCS3200 Color Sensor	VCC, GND	3V, G
Servo Motor	Signal	D8
Servo Motor	VCC, GND	5V, G
RED LED	+ (Anode)	D3
RED LED	-(Cathode)	G
GREEN LED	+ (Anode)	D4
GREEN LED	-(Cathode)	G

CIRCUIT DESIGN



ALGORITHM



CODING

```
#include <Servo.h>
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClientSecure.h>

// =====
// WIFI SETTINGS
// =====

const char* WIFI_SSID = "P4";
const char* WIFI_PASSWORD = "1234567890";

// =====
// THINGSPEAK WRITE API KEY
// =====

const char* THINGSPEAK_API_KEY = "N59W7SM6U0LGRU23";

// =====
// ESP8266 NODEMCU PIN ASSIGNMENTS
// =====

// TCS3200
const int S0_PIN = 5;      // D1
const int S1_PIN = 4;      // D2
const int S2_PIN = 14;     // D5
const int S3_PIN = 12;     // D6
const int OUT_PIN = 13;    // D7

// LEDs
const int RED_LED_PIN = 0;  // D3
const int GREEN_LED_PIN = 2; // D4

// Servo
const int SERVO_PIN = 15;   // D8

Servo sortingServo;

// =====
// SERVO POSITIONS
// =====

const int SERVO_CENTER = 90;
const int SERVO_RED = 30;
const int SERVO_GREEN = 150;
```

```

// =====
// PRODUCT COUNTERS
// =====

unsigned long redCount = 0;
unsigned long greenCount = 0;

// =====
// DETECTED COLOR
//
// 0 = No valid color
// 1 = Red
// 2 = Green
// =====

int detectedColor = 0;

// =====
// THINGSPEAK UPDATE TIMER
// =====

unsigned long lastThingSpeakUpdate = 0;

const unsigned long THINGSPEAK_INTERVAL = 15000;

// =====
// READ TCS3200
// =====

unsigned long readTCS3200(int s2State, int s3State)
{
    digitalWrite(S2_PIN, s2State);
    digitalWrite(S3_PIN, s3State);

    delay(50);

    unsigned long pulseWidth =
        pulseIn(OUT_PIN, LOW, 100000);

    return pulseWidth;
}

// =====

```

```

// CONNECT TO WIFI
// =====

void connectWiFi()
{
    Serial.println();
    Serial.print("Connecting to WiFi");

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    int attempts = 0;

    while (WiFi.status() != WL_CONNECTED && attempts < 30)
    {
        delay(500);
        Serial.print(".");
        attempts++;
    }

    Serial.println();

    if (WiFi.status() == WL_CONNECTED)
    {
        Serial.println("WiFi Connected!");

        Serial.print("IP Address: ");
        Serial.println(WiFi.localIP());
    }
    else
    {
        Serial.println("WiFi Connection Failed!");
    }
}

// =====
// SEND DATA TO THINGSPEAK
// =====

void sendToThingSpeak()
{
    // -----
    // CHECK WIFI
    // -----

    if (WiFi.status() != WL_CONNECTED)
    {
        Serial.println("WiFi disconnected!");
        connectWiFi();
    }
}

```

```

}

if (WiFi.status() == WL_CONNECTED)
{
    WiFiClientSecure client;

    // For ESP8266 testing
    client.setInsecure();

    HTTPClient http;

    // -----
    // CALCULATE TOTAL
    // -----

    unsigned long totalSorted =
        redCount + greenCount;

    // -----
    // THINGSPEAK SERVER URL
    // -----

    String url =
        "https://api.thingspeak.com/update.json?api_key=";

    url += THINGSPEAK_API_KEY;

    // -----
    // FIELD 1
    // 1 = RED
    // 2 = GREEN
    // -----

    url += "&field1=";
    url += String(detectedColor);

    // -----
    // FIELD 2
    // RED COUNT
    // -----

    url += "&field2=";
    url += String(redCount);

```

```

// -----
// FIELD 3
// GREEN COUNT
// -----

url += "&field3=";
url += String(greenCount);

// -----
// FIELD 4
// TOTAL SORTED
// -----

url += "&field4=";
url += String(totalSorted);

// -----
// SERIAL MONITOR
// -----

Serial.println();
Serial.println("=====");
Serial.println("THINGSPEAK UPDATE");
Serial.println("=====");

Serial.print("Detected Color : ");
Serial.println(detectedColor);

Serial.print("Red Count      : ");
Serial.println(redCount);

Serial.print("Green Count    : ");
Serial.println(greenCount);

Serial.print("Total Sorted   : ");
Serial.println(totalSorted);

Serial.println();
Serial.println("Connecting to ThingSpeak...");

// -----
// HTTP REQUEST
// -----

if (http.begin(client, url))
{

```

```

int httpCode = http.GET();

// -----
// SERVER RESPONSE
// -----

Serial.print("HTTP Response Code: ");
Serial.println(httpCode);

if (httpCode > 0)
{
    String response =
        http.getString();

    Serial.print("ThingSpeak Response: ");
    Serial.println(response);

    // -----
    // SUCCESS
    // -----

    if (response.indexOf("\"entry_id\"") >= 0)
    {
        Serial.println();
        Serial.println(
            ">>> THINGSPEAK UPDATE SUCCESSFUL <<<"
        );
    }
    else if (response == "0")
    {
        Serial.println();
        Serial.println(
            ">>> THINGSPEAK UPDATE FAILED <<<"
        );
        Serial.println(
            "Check Write API Key and Channel."
        );
    }
    else
    {
        Serial.println(
            "ThingSpeak returned a response."
        );
    }
}
else

```

```

        {
            Serial.print("HTTP Error: ");
            Serial.println(
                http.errorToString(httpCode)
            );
        }

        http.end();
    }
    else
    {
        Serial.println(
            "Unable to connect to ThingSpeak server!"
        );
    }

    Serial.println("=====");
}
}

// =====
// SETUP
// =====

void setup()
{
    Serial.begin(115200);

    // =====
    // TCS3200
    // =====

    pinMode(S0_PIN, OUTPUT);
    pinMode(S1_PIN, OUTPUT);

    pinMode(S2_PIN, OUTPUT);
    pinMode(S3_PIN, OUTPUT);

    pinMode(OUT_PIN, INPUT);

    // =====
    // LEDs
    // =====

    pinMode(RED_LED_PIN, OUTPUT);
    pinMode(GREEN_LED_PIN, OUTPUT);

```

```
digitalWrite(RED_LED_PIN, LOW);
digitalWrite(GREEN_LED_PIN, LOW);

// =====
// TCS3200 FREQUENCY SCALING
// =====

digitalWrite(S0_PIN, HIGH);
digitalWrite(S1_PIN, LOW);

// =====
// SERVO
// =====

sortingServo.attach(SERVO_PIN);

sortingServo.write(SERVO_CENTER);

delay(1000);

// =====
// WIFI
// =====

connectWiFi();

// =====
// STARTUP MESSAGE
// =====

Serial.println();
Serial.println("=====");
Serial.println(" AUTOMATED COLOR SORTING");
Serial.println("=====");
Serial.println("TCS3200      : READY");
Serial.println("Servo        : READY");
Serial.println("WiFi         : READY");
Serial.println("ThingSpeak   : READY");
Serial.println("=====");
Serial.println("SYSTEM READY");
Serial.println();
```

```

// =====
// INITIAL THINGSPEAK TEST
// =====

if (WiFi.status() == WL_CONNECTED)
{
    Serial.println("Testing ThingSpeak connection...");

    sendToThingSpeak();

    lastThingSpeakUpdate = millis();
}
}

// =====
// MAIN LOOP
// =====

void loop()
{
    unsigned long red;
    unsigned long green;

    // =====
    // READ RED
    // S2 = LOW
    // S3 = LOW
    // =====

    red = readTCS3200(LOW, LOW);

    // =====
    // READ GREEN
    // S2 = HIGH
    // S3 = HIGH
    // =====

    green = readTCS3200(HIGH, HIGH);

    // =====
    // DISPLAY SENSOR VALUES
    // =====

    Serial.println("-----");

```

```
Serial.print("RED   = ");
Serial.println(red);

Serial.print("GREEN = ");
Serial.println(green);

// =====
// INVALID SENSOR READING
// =====

if (red == 0 || green == 0)
{
    Serial.println("No valid sensor reading");

    detectedColor = 0;

    digitalWrite(RED_LED_PIN, LOW);
    digitalWrite(GREEN_LED_PIN, LOW);

    sortingServo.write(SERVO_CENTER);

    delay(500);
}

// =====
// RED OBJECT
// =====

else if (red < green)
{
    Serial.println("RED OBJECT DETECTED");

    detectedColor = 1;

    redCount++;

    // Red LED
    digitalWrite(RED_LED_PIN, HIGH);
    digitalWrite(GREEN_LED_PIN, LOW);

    // Servo → Red position
    sortingServo.write(SERVO_RED);

    delay(1000);
```

```

// Servo → Center
sortingServo.write(SERVO_CENTER);

delay(500);
}

// =====
// GREEN OBJECT
// =====

else if (green < red)
{
    Serial.println("GREEN OBJECT DETECTED");

    detectedColor = 2;

    greenCount++;

    // Green LED
    digitalWrite(RED_LED_PIN, LOW);
    digitalWrite(GREEN_LED_PIN, HIGH);

    // Servo → Green position
    sortingServo.write(SERVO_GREEN);

    delay(1000);

    // Servo → Center
    sortingServo.write(SERVO_CENTER);

    delay(500);
}

// =====
// EQUAL VALUES
// =====

else
{
    Serial.println("COLOR NOT DETECTED");

    detectedColor = 0;

    digitalWrite(RED_LED_PIN, LOW);

```

```

    digitalWrite(GREEN_LED_PIN, LOW);

    sortingServo.write(SERVO_CENTER);

    delay(500);
}

// =====
// THINGSPEAK UPDATE EVERY 15 SECONDS
// =====

if (millis() - lastThingSpeakUpdate >=
    THINGSPEAK_INTERVAL)
{
    Serial.println();
    Serial.println(
        "Uploading data to ThingSpeak..."
    );

    sendToThingSpeak();

    lastThingSpeakUpdate = millis();
}
}

```

SYSTEM WORKING

The Automated Color-Based Product Sorting System works by using the TCS3200 color sensor to detect the color of a product and an ESP8266 NodeMCU to process the sensor information and control the sorting mechanism. When the system is powered ON, the ESP8266 initializes the TCS3200 sensor, servo motor, and LED indicators. The servo motor is initially positioned at the center position.

A product is manually placed in front of the TCS3200 color sensor. The sensor detects the light reflected from the surface of the product and produces frequency signals corresponding to its color components. The ESP8266 reads these signals and compares the red and green sensor values to determine the product color.

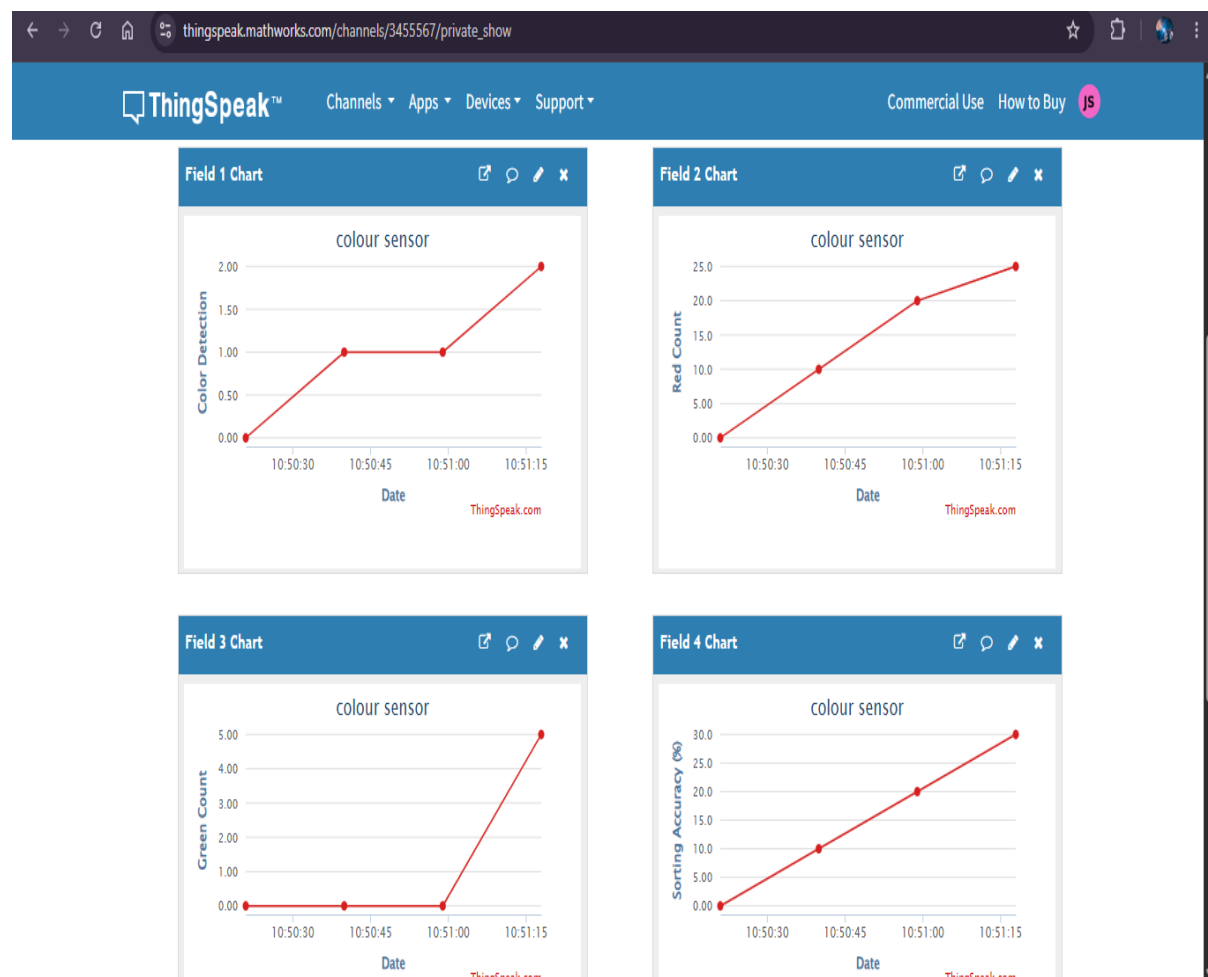
If a red product is detected, the ESP8266 turns ON the red LED and rotates the servo motor to the predefined red sorting position. After a short delay, the servo returns to its center position. Similarly, if a green product is detected, the green LED turns ON and the servo rotates to the predefined green sorting position before returning to the center position.

If the sensor does not receive a valid reading or the color cannot be identified, both LEDs remain OFF and the servo stays at the center position. The process is repeated continuously for every product placed in front of the sensor.

The ESP8266 also provides Wi-Fi connectivity, which can be used to transmit information such as detected color, product count, and sorting status to an IoT platform such as ThingSpeak. This allows the system to be monitored remotely and provides a foundation for smart manufacturing applications.

Thus, the complete working sequence is:

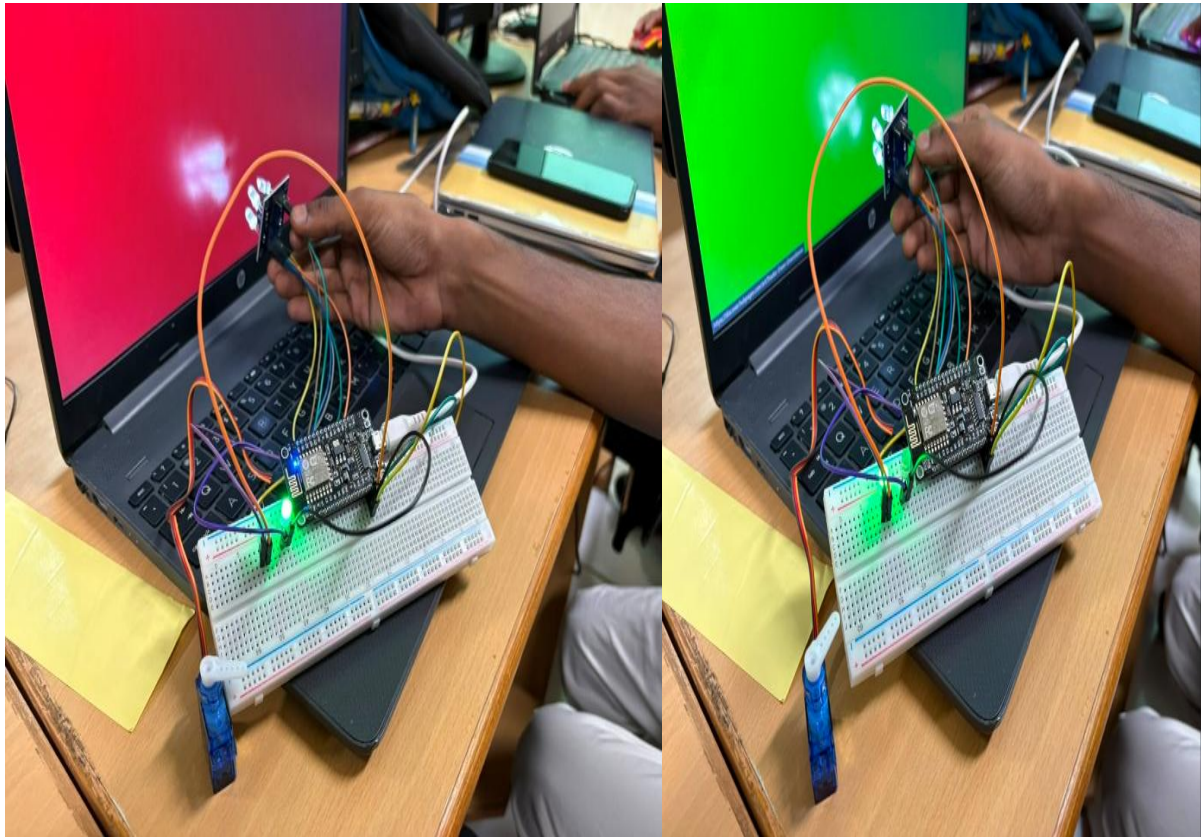
Product Placement → Color Detection → TCS3200 Sensor → ESP8266 Processing → Color Identification → LED Indication → Servo Movement → Sorting → Repeat.



BILL OF MATERIALS

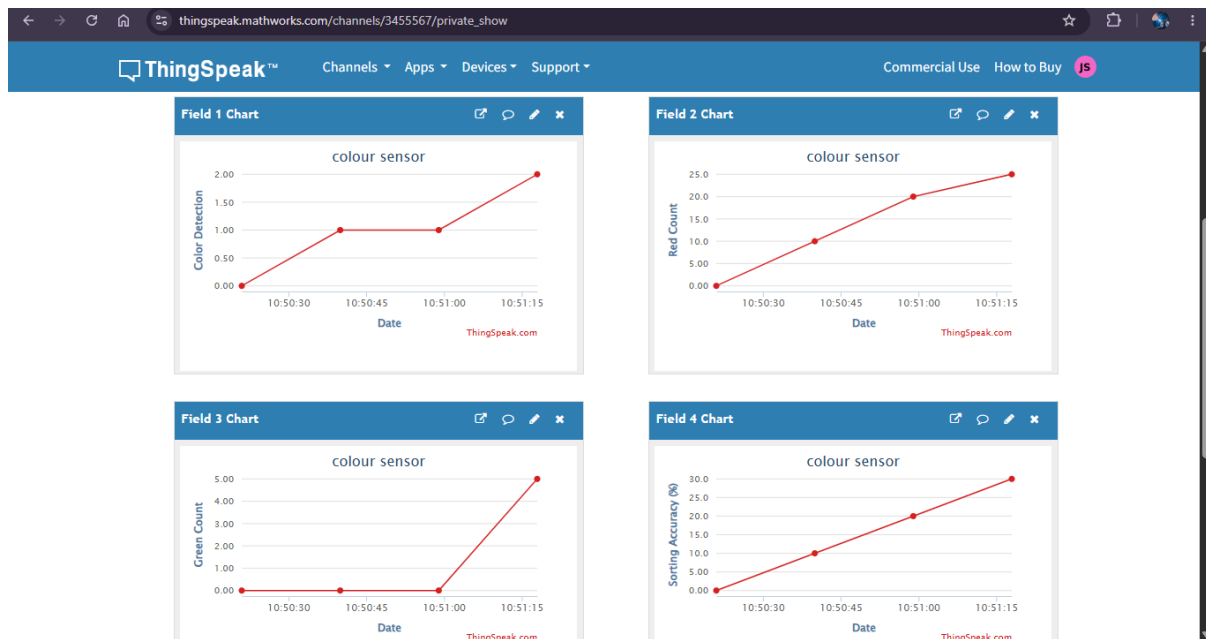
COMPONENTS	QUANTITY
ESP8266 Node MCU	1
TCS3200 Color Sensor	1
MQ135 Air Quality Servo Motor Sensor	1
LED Indicators	2
Breadboard	1
Jumper Wires	1 Set

PROTOTYPE IMPLEMENTATION



The prototype is implemented using an ESP8266 NodeMCU, TCS3200 color sensor, servo motor, red and green LEDs, jumper wires, and a power supply. The ESP8266 acts as the main controller and is programmed using Arduino IDE. The TCS3200 detects the color of a product and sends the sensor readings to the ESP8266 for processing. Based on the detected color, the ESP8266 activates the corresponding LED and rotates the servo motor to a predefined sorting position. Since a conveyor is not available, products are placed manually in front of the sensor. The servo then returns to its center position after sorting. The ESP8266 can also use Wi-Fi to transmit color and sorting information to an IoT platform for monitoring.

RESULT & PERFORMANCE ANALYSIS



The developed prototype successfully detected and classified products based on their color using the TCS3200 color sensor and ESP8266 NodeMCU. When a red product was placed in front of the sensor, the system activated the red LED and moved the servo motor to the predefined red sorting position. Similarly, for a green product, the green LED was activated and the servo moved to the corresponding position.

The system provided reliable color detection under controlled lighting conditions and demonstrated the expected sorting response. The ESP8266 successfully processed the sensor readings and controlled the LED indicators and servo motor. Since the conveyor was not included in the prototype, products were placed manually in front of the sensor for testing.

The performance of the system mainly depends on sensor calibration, lighting conditions, distance between the sensor and product, and the color and surface of the object. With proper calibration and stable lighting, the system can provide consistent results. The Wi-Fi capability of the ESP8266 also provides scope for monitoring detected colors and sorting information through an IoT platform.

Overall, the prototype successfully demonstrates the basic concept of automated color detection and sorting and shows its potential for application in manufacturing and smart automation systems.